

APEX AI: Detailed Technical Documentation

1. Overview

APEX AI stands for **AI Powered Masters Program Explorer**. It is a full-stack educational consultant application designed to help students discover suitable Masters programs through natural conversation.[\[ppl-ai-file-upload.s3.amazonaws\]](#)

Instead of forcing users to fill out a static form, APEX AI chats with them and extracts three core preferences: **domain of interest**, **preferred country or countries**, and **tuition budget**. Once those inputs are available, the system generates personalized program recommendations, a learning roadmap, and a final motivational summary.[\[ppl-ai-file-upload.s3.amazonaws\]](#)

The system is built to feel like an intelligent academic advisor rather than a generic chatbot. It uses structured AI workflows, Markdown-based output, and a polished frontend to make the experience both informative and visually clear.[\[ppl-ai-file-upload.s3.amazonaws\]](#)

2. What APEX AI Does

APEX AI is built around three major user outcomes:[\[ppl-ai-file-upload.s3.amazonaws\]](#)

- It discovers relevant Masters programs based on a student's profile.
- It generates a domain-specific learning roadmap.
- It provides a final synthesized summary that connects the recommendations and roadmap into a cohesive guide.

The first stage is conversational discovery. The AI keeps asking and extracting until it has enough information to move forward.[\[ppl-ai-file-upload.s3.amazonaws\]](#)

The second stage is generation. Once the required fields are confirmed, specialized backend workflows create structured output for programs, learning steps, and a summary.[\[ppl-ai-file-upload.s3.amazonaws\]](#)

3. How It Works

APEX follows a clear end-to-end flow:[\[ppl-ai-file-upload.s3.amazonaws\]](#)

1. The app launches with a loading screen.
2. It transitions to a 3D globe-style landing page.
3. The chat interface opens.
4. The user sends a message.

5. The backend `/api/chat` endpoint extracts structured preferences while generating a reply.
6. If any of the three required fields are missing, the AI asks for the missing ones.
7. When all three fields are present, the backend triggers three workflows in sequence:
 - Top Programs
 - Roadmap
 - Summary
8. The frontend renders the outputs as Markdown.

This creates a simulated agentic loop: the system perceives user input, reasons about completeness, then acts by launching specialized workflows.[\[ppl-ai-file-upload.s3.amazonaws\]](#)

4. AI Workflow Logic

The chat extraction flow is the core of the system. It receives the conversation history and the latest user message, then produces both a conversational response and a structured JSON-like extraction containing domain, country, and budget.[\[ppl-ai-file-upload.s3.amazonaws\]](#)

Once the required information is complete, the backend runs the next workflows in order:[\[ppl-ai-file-upload.s3.amazonaws\]](#)

- **Top Programs:** generates the top 5 Masters program recommendations.
- **Roadmap:** generates a structured learning path for the selected domain.
- **Summary:** combines the outputs into a motivational final overview.

This design separates conversation handling from content generation. That makes the system easier to maintain and improves the consistency of outputs.[\[ppl-ai-file-upload.s3.amazonaws\]](#)

5. AI Chains Used

All chains are located in `backend/chains/workflow.py`, and they use prompt templates from `backend/prompts/templates.py`. The chains are built with LangChain Expression Language using the pattern:[\[ppl-ai-file-upload.s3.amazonaws\]](#)

`prompt | llm | parser` [\[ppl-ai-file-upload.s3.amazonaws\]](#)

5.1 Chat Extraction Chain

This chain reads conversation history and the latest user message. Its job is to generate a friendly reply while extracting:[\[ppl-ai-file-upload.s3.amazonaws\]](#)

- Domain of interest
- Preferred country/countries

- Tuition budget

If one or more fields are missing, the chain returns a natural follow-up asking for the missing information.[[ppl-ai-file-upload.s3.amazonaws](#)]

5.2 Top Programs Chain

This chain generates five Masters recommendations based on the user's preferences. The recommendations are scored on:[[ppl-ai-file-upload.s3.amazonaws](#)]

- Reputation
- Research
- Industry exposure
- Employability

It uses a two-step pipeline:[[ppl-ai-file-upload.s3.amazonaws](#)]

1. Generate raw recommendations.
2. Refine the output into polished Markdown.

That second pass improves readability and presentation quality.[[ppl-ai-file-upload.s3.amazonaws](#)]

5.3 Roadmap Generator Chain

This chain creates a domain-specific roadmap. The roadmap is organized into seven categories:[[ppl-ai-file-upload.s3.amazonaws](#)]

- Foundations
- Core Concepts
- Tools and Technologies
- Advanced Topics
- Projects
- Research
- Career Prep

The goal is to show students what they should learn and in what direction they should grow.[[ppl-ai-file-upload.s3.amazonaws](#)]

5.4 Summary Chain

This final chain synthesizes the recommendations and roadmap into one cohesive conclusion. It is designed to feel encouraging, useful, and aligned with the student's goals.[[ppl-ai-file-upload.s3.amazonaws](#)]

6. Core Concepts Used

APEX AI uses several important AI engineering concepts.[\[ppl-ai-file-upload.s3.amazonaws\]](#)

Structured Output in Prompts

The system asks the model to return both a natural-language reply and structured data. This is useful because it lets the app keep a conversation going while still extracting machine-readable fields.[\[ppl-ai-file-upload.s3.amazonaws\]](#)

LangChain LCEL Pipelines

The workflows are composed as LangChain Expression Language pipelines. This makes the logic modular and easy to test, since each stage can be swapped or improved independently.[\[ppl-ai-file-upload.s3.amazonaws\]](#)

Frontend State Management

The React app uses local state to manage the loading, landing, and chat phases. It does not rely on Redux, which keeps the client architecture lightweight.[\[ppl-ai-file-upload.s3.amazonaws\]](#)

Simulated Agent Behavior

The application behaves like an agent by extracting context, checking completeness, and triggering actions. It is not described as a full retrieval-based agent, but it does simulate a Plan-Act-Observe style flow.[\[ppl-ai-file-upload.s3.amazonaws\]](#)

Two-Pass Refinement

The Top Programs workflow uses a generation pass followed by a refinement pass. This helps ensure the final Markdown is polished and scannable.[\[ppl-ai-file-upload.s3.amazonaws\]](#)

LLM as the Knowledge Source

The system relies on Gemini's internal knowledge rather than an external RAG pipeline. This keeps the architecture simpler while still supporting advisory-style output.[\[ppl-ai-file-upload.s3.amazonaws\]](#)

7. System Architecture

APEX is a two-tier application. The frontend is a React app, and the backend is a FastAPI service that orchestrates LangChain workflows powered by Google Gemini.[\[ppl-ai-file-upload.s3.amazonaws\]](#)

Frontend Layer

The frontend handles:

- Loading animation
- Landing page
- Chat interface
- Markdown rendering

It uses React 19, Vite, Tailwind CSS v4, Axios, react-markdown, remark-gfm, lucide-react, react-globe.gl, and three.js.[\[ppl-ai-file-upload.s3.amazonaws\]](#)

Backend Layer

The backend handles:

- API routing
- Request validation
- Chain orchestration
- LLM calls
- Structured response creation

It uses FastAPI, Uvicorn, Pydantic, LangChain LCEL, Google Gemini 2.5 Flash, and python-dotenv.[\[ppl-ai-file-upload.s3.amazonaws\]](#)

AI Layer

The AI layer includes:

- LangChain pipelines
- Prompt templates
- Output parsers
- Gemini 2.5 Flash
- Workflow orchestration in `workflow.py`.[\[ppl-ai-file-upload.s3.amazonaws\]](#)

8. Tech Stack

Layer	Tools	Purpose
Backend	FastAPI, Uvicorn, Pydantic	API handling, validation, async serving [ppl-ai-file-upload.s3.amazonaws]
AI Orchestration	LangChain LCEL	Chain composition and workflow management [ppl-ai-file-upload.s3.amazonaws]
LLM	Google Gemini 2.5 Flash	Core language generation for all chains [ppl-ai-file-upload.s3.amazonaws]

Config	python-dotenv	Environment variable loading [ppl-ai-file-upload.s3.amazonaws]
Frontend	React 19, Vite	UI and build tooling [ppl-ai-file-upload.s3.amazonaws]
Styling	Tailwind CSS v4	Visual design and responsiveness [ppl-ai-file-upload.s3.amazonaws]
UI Helpers	Axios, react-markdown, remark-gfm	API requests and Markdown rendering [ppl-ai-file-upload.s3.amazonaws]
Visuals	react-globe.gl, three.js	Landing-page 3D experience [ppl-ai-file-upload.s3.amazonaws]

9. API Endpoints

All endpoints are defined in `backend/routes/api.py` and registered in `backend/main.py`. The FastAPI server runs on `localhost:8000` by default.[\[ppl-ai-file-upload.s3.amazonaws\]](#)

Endpoint	Purpose	Input	Output
POST <code>/api/chat</code>	Conversational extraction and reply generation	History + latest message	Reply plus extracted fields [ppl-ai-file-upload.s3.amazonaws]
POST <code>/api/top-programs</code>	Generate top 5 Masters recommendations	Domain, country, budget	Markdown program list [ppl-ai-file-upload.s3.amazonaws]
POST <code>/api/generate-roadmap</code>	Generate learning roadmap	Domain	Markdown roadmap [ppl-ai-file-upload.s3.amazonaws]
POST <code>/api/summary</code>	Generate final synthesis	Programs + roadmap	Markdown summary [ppl-ai-file-upload.s3.amazonaws]

10. Project File Structure

APEX uses a clean separation between backend and frontend responsibilities.[\[ppl-ai-file-upload.s3.amazonaws\]](#)

text

```
backend/
├── main.py
├── routes/
│   └── api.py
├── chains/
│   └── workflow.py
├── prompts/
│   └── templates.py
├── models/
│   └── schemas.py
├── services/
│   └── llmservice.py
└── .env

frontend/
├── src/
│   ├── App.jsx
│   ├── components/
│   │   ├── Loading.jsx
│   │   ├── Landing.jsx
│   │   └── ChatInterface.jsx
│   ├── api/
│   └── styles/
└── public/
```

This structure makes the project easy to understand and extend. Backend logic stays isolated from UI logic, and the chain definitions remain separate from the prompt templates.[\[ppl-ai-file-upload.s3.amazonaws\]](#)

11. Data Flow

The data flow begins with user input and ends with a rendered response in the browser.[\[ppl-ai-file-upload.s3.amazonaws\]](#)

1. The user submits a chat message.
2. The frontend sends the request to the backend.
3. The backend extracts domain, country, and budget.
4. The system checks whether all required fields are available.
5. If not, it asks for missing information.
6. If yes, it triggers the three generation workflows.
7. The results are returned as Markdown.
8. The frontend renders them visually.

A typical extracted preference payload looks like this:[\[ppl-ai-file-upload.s3.amazonaws\]](#)

```
json
{
  "domain": "Machine Learning",
  "country": "Germany",
  "budget": "under 20,000 EUR/year"
}
```

A chat request payload looks like this:[\[ppl-ai-file-upload.s3.amazonaws\]](#)

```
json
{
  "history": [
    {"role": "user", "content": "..."},
    {"role": "ai", "content": "..."}
  ],
  "message": "I want to study ML in Germany"
}
```

12. Why The Design Works

APEX is effective because it balances simplicity and capability. It does not overcomplicate the architecture with unnecessary retrieval systems or external databases.[\[ppl-ai-file-upload.s3.amazonaws\]](#)

The separation into extraction, recommendation, roadmap, and summary makes the system easy to reason about. Each chain has one job, which improves maintainability and output quality.[\[ppl-ai-file-upload.s3.amazonaws\]](#)

The frontend presentation also matters. Because the AI returns Markdown, the app can show structured, readable, and professional-looking responses.[\[ppl-ai-file-upload.s3.amazonaws\]](#)

13. Conclusion

APEX AI is not just a chatbot; it is a guided academic decision-support system. It uses conversational extraction, chained LLM workflows, and a modern full-stack architecture to help students identify Masters programs and prepare for them effectively.[\[ppl-ai-file-upload.s3.amazonaws\]](#)

If you want this as a true PDF file, the fastest path is to copy this document into Word or Google Docs and export it as PDF.